Project Introduction :

The Library Management System project is developed to improve the efficiency and organization of library operations.
This system provides a comprehensive solution for managing books, users, and lending activities within a library environment.
By utilizing a well-structured relational database, the system ensures accurate tracking of available books, user registrations, book loans, and returns.
The main objective of the project is to streamline routine tasks, reduce manual errors, and enhance user experience for both library staff and members.
Overall, the Library Management System plays a critical role in modernizing library services and supporting better information management.

System Description:

The Library Management System is a database-driven application designed to handle the essential functions of a library.
It manages book records, member information, and borrowing and returning processes.
The system allows librarians to add, update, and delete books and users easily.
It also tracks which books are borrowed and when they are due for return.
Overall, the system improves the organization, efficiency, and reliability of library operations.

The code:

```sql
DROP TABLE Books CASCADE CONSTRAINTS;
CREATE TABLE Books (
    book_id NUMBER PRIMARY KEY,
    title VARCHAR2(50) NOT NULL,
    author VARCHAR2(50) NOT NULL,
    quantity NUMBER
);

DROP TABLE Members CASCADE CONSTRAINTS;
CREATE TABLE Members (
    member_id NUMBER PRIMARY KEY,
    f_name VARCHAR2(10),
    L_name VARCHAR2(20),
    email VARCHAR2(40)
);

DROP TABLE Borrowing CASCADE CONSTRAINTS;
CREATE TABLE Borrowing (
borrow_id number PRIMARY key,
member_id number,
book_id number,
borrowdate date,
duedate date,
returndate date,
CONSTRAINT fk_borrow_member FOREIGN key (member_id) REFERENCES Members(member_id),
CONSTRAINT fk_borrow_book FOREIGN key (book_id) REFERENCES books(book_id)
);

DROP TABLE Users CASCADE CONSTRAINTS;
CREATE TABLE Users (
    userId NUMBER PRIMARY KEY,
    username VARCHAR2(50),
    password VARCHAR2(10) NOT NULL,
    role VARCHAR2(50)
);
DROP TABLE payment CASCADE CONSTRAINTS;
Create TABLE payment(
payment_id number PRIMARY key,
borrow_id number,
member_id number,
deptamount number(10,2),
paymentdate date,
status VARCHAR2(10),
CONSTRAINT fk_payment_borrow FOREIGN key (borrow_id) REFERENCES Borrowing(borrow_id),
CONSTRAINT fk_payment_member FOREIGN key (member_id) REFERENCES Members(member_id)
);

INSERT INTO Books VALUES (100, 'The black cat', 'jackson', 5);
INSERT INTO Books VALUES (101, 'think and grow Rich', 'napoleon hill', 3);
INSERT INTO Books VALUES (102, 'the art of happiness', 'dalai lama and howard cutler', 4);
INSERT INTO Books VALUES (103, '1984', 'George Orwell', 2);
INSERT INTO Books VALUES (104, 'To Kill a Mockingbird', 'Harper Lee', 4);
INSERT INTO Books VALUES (105, 'Sapiens', 'Yuval Noah Harari', 5);
INSERT INTO Books VALUES (106, 'Atomic Habits', 'James Clear', 3);
INSERT INTO Books VALUES (107, 'The Power of Now', 'Eckhart Tolle', 10);

INSERT INTO Members VALUES (01, 'Ahmed', 'Ali', 'ahmed@gmail.com');
INSERT INTO Members VALUES (02, 'jana', 'Ibrahim', 'jana@gmail.com');
INSERT INTO Members VALUES (03, 'Sara', 'Mohammed', 'sara@gmail.com');
INSERT INTO Members VALUES (04, 'Faisal', 'Salem', 'faisal@hotmail.com');

INSERT INTO Borrowing VALUES (1134, 01, 100, DATE '2025-04-01', DATE '2025-04-10', NULL);
INSERT INTO Borrowing VALUES (1156, 02, 101, DATE '2025-04-02', DATE '2025-04-12', DATE '2025-04-11');
INSERT INTO Borrowing VALUES (1178, 03, 102, DATE '2025-04-03', DATE '2025-04-15', NULL);
INSERT INTO Borrowing VALUES (1190, 04, 103, DATE '2025-04-04', DATE '2025-04-14', DATE '2025-04-13');


INSERT INTO Users VALUES (1, 'Wajd', 'W01', 'Admin');
INSERT INTO Users VALUES (2, 'Lina', 'L02', 'Moderator');
INSERT INTO Users VALUES (3, 'Rawda', 'Raw03', 'Member');
INSERT INTO Users VALUES (4, 'Reema', 'R04', 'Guest');

INSERT INTO payment VALUES (704, 1156, 02, 0.00, DATE '2025-04-11', 'Paid');
INSERT INTO payment VALUES (805, 1134, 01, 20.00, NULL, 'Unpaid');
INSERT INTO payment VALUES (910, 1178, 03, 50.00, DATE '2025-04-10', 'Paid');
INSERT INTO payment VALUES (1023, 1190, 04, 35.00, NULL, 'Unpaid');
```

```sql
--TRIGGER--

create or replace TRIGGER unavailableborrow
BEFORE INSERT on Borrowing
for EACH row
DECLARE
t_quantity number;
BEGIN
select quantity into t_quantity
from Books
where book_id=:new.book_id;
if t_quantity<=0 THEN
RAISE_APPLICATION_ERROR(-20001,'the book is not available!');
end if;
end;
/

--trying TRIGGER
update books set quantity=0 where book_id=100;
 INSERT INTO Borrowing (borrow_id, member_id, book_id, borrowdate, duedate, returndate)
 VALUES (999, 101, 100, SYSDATE, SYSDATE+7, NULL);


--PROCEDURE--

create or REPLACE PROCEDURE returnbook(p_borrow_id in number)IS
BEGIN
update Borrowing
set returndate=SYSDATE
where borrow_id=p_borrow_id;
DBMS_OUTPUT.PUT_LINE('the book has been returned');
end;
/

--trying PROCEDURE
set serveroutput on;
declare
v_borrow_id number;
BEGIN
v_borrow_id:=&borrow_id;
returnbook(v_borrow_id);
end;
/

--FUNCTION--

create or replace FUNCTION calculatedept(p_borrow_id in NUMBER)
RETURN number is
v_due date;
v_return date;
v_dept number:=0;
BEGIN
select duedate,returndate
into v_due,v_return
from Borrowing
where borrow_id=p_borrow_id;
if v_return >v_due then
v_dept:=(v_return - v_due)*2;
end if;
return v_dept;
end;
/

--trying function
set serveroutput on;
declare
v_borrow_id number;
v_dept number;
BEGIN
v_borrow_id:=&borrow_id;
v_dept := calculatedept(v_borrow_id);
DBMS_OUTPUT.PUT_LINE('calculated dept amount: '||v_dept);
end;
/
```

```sql
--QUERIES--
select b.book_id,b.title,count(*) as timesborrowed
from Borrowing b1
join books b on b1.book_id=b.book_id
GROUP by b.book_id,b.title
ORDER by timesborrowed DESC;

--before optimization
select * from books
where author='George Orwell';
--after optimization
select title
from books
where author='George Orwell';

--before optimization
explain plan for
select m.member_id,m.f_name,m.L_name,b.duedate
from Members m
join Borrowing b on m.member_id=b.member_id
where b.duedate<SYSDATE and b.returndate is null;
select * from table(dbms_xplan.display);
--after optimization
explain plan for
create index idx_borrowing_bookid on Borrowing(book_id);
SELECT m.member_id, m.f_name, m.L_name
FROM Members m
WHERE EXISTS (
  select * FROM Borrowing b
   WHERE b.member_id = m.member_id
     AND b.duedate < SYSDATE
     AND b.returndate IS NULL
);
select * from table(dbms_xplan.display);


DROP USER wajd CASCADE;
DROP USER Lina CASCADE;
DROP USER Reema CASCADE;
DROP USER Rawda CASCADE;

DROP ROLE Admin;
DROP ROLE Moderator;
DROP ROLE Guest;
DROP ROLE Member;


--ADMIN--
--create USER , create Role
CREATE USER wajd IDENTIFIED BY W01;
CREATE ROLE Admin;

--grant privileges , assign role to user
GRANT ALL PRIVILEGES ON Books TO Admin;
GRANT ALL PRIVILEGES ON Members TO Admin;
GRANT ALL PRIVILEGES ON Borrowing TO Admin;
GRANT ALL PRIVILEGES ON Users TO Admin;
GRANT CREATE USER, ALTER USER, DROP USER, CREATE SESSION TO Admin;
GRANT Admin TO wajd;
--MODERATOR--
--create USER , create Role

CREATE USER Lina IDENTIFIED BY L02;
CREATE ROLE Moderator;

--grant privileges , assign role to user
GRANT SELECT,INSERT ,UPDATE ,DELETE ON Books TO Moderator;
GRANT SELECT, UPDATE ON Members TO Moderator;
GRANT SELECT ON Borrowing TO Moderator;
GRANT Moderator TO Lina;


--MEMBER--
--create USER , create Role
CREATE USER Rawda IDENTIFIED BY Raw03;
CREATE ROLE Member;

--grant privileges , assign role to user
GRANT SELECT ON Books TO Member;
GRANT SELECT, INSERT, UPDATE ON Borrowing TO Member;
GRANT Member TO Rawda;

--GUEST--
--create USER , create Role
CREATE USER Reema IDENTIFIED BY R04;
CREATE ROLE Guest;

--grant privileges , assign role to user
GRANT SELECT ON Books TO Guest;
GRANT SELECT ON Borrowing TO Guest;
GRANT Guest TO Reema;

```

The output:

Procedure to Return a Book:

```
SQL> --PROCEDURE--
SQL>
SQL> create or REPLACE PROCEDURE returnbook(p_borrow_id in number)IS
  2  BEGIN
  3  update Borrowing
  4  set returndate=SYSDATE
  5  where borrow_id=p_borrow_id;
  6  DBMS_OUTPUT.PUT_LINE('the book has been returned');
  7  end;
  8  /

Procedure created.

SQL>
SQL> --trying PROCEDURE
SQL> set serveroutput on;
SQL> declare
  2  v_borrow_id number;
  3  BEGIN
  4  v_borrow_id:=&borrow_id;
  5  returnbook(v_borrow_id);
  6  end;
  7  /
Enter value for borrow_id: 1134
old    4: v_borrow_id:=&borrow_id;
new    4: v_borrow_id:=1134;
the book has been returned

PL/SQL procedure successfully completed.
```

Function to Calculate Debt Amount :

```
SQL> --FUNCTION--
SQL>
SQL> create or replace FUNCTION calculatedept(p_borrow_id in NUMBER)
  2  RETURN number is
  3  v_due date;
  4  v_return date;
  5  v_dept number:=0;
  6  BEGIN
  7  select duedate,returndate
  8  into v_due,v_return
  9  from Borrowing
 10  where borrow_id=p_borrow_id;
 11  if v_return >v_due then
 12  v_dept:=(v_return - v_due)*2;
 13  end if;
 14  return v_dept;
 15  end;
 16  /

Function created.

SQL> --trying function
SQL> set serveroutput on;
SQL> declare
  2  v_borrow_id number;
  3  v_dept number;
  4  BEGIN
  5  v_borrow_id:=&borrow_id;
  6  v_dept := calculatedept(v_borrow_id);
  7  DBMS_OUTPUT.PUT_LINE('calculated dept amount: '||v_dept);
  8  end;
  9  /
Enter value for borrow_id: 1134
old    5: v_borrow_id:=&borrow_id;
new    5: v_borrow_id:=1134;
calculated dept amount: 33.309375

PL/SQL procedure successfully completed.
```

Trigger to Prevent Borrowing Unavailable Books :

```
SQL> --TRIGGER--
SQL>
SQL> create or replace TRIGGER unavailableborrow
  2  BEFORE INSERT on Borrowing
  3  for EACH row
  4  DECLARE
  5  t_quantity number;
  6  BEGIN
  7  select quantity into t_quantity
  8  from Books
  9  where book_id=:new.book_id;
 10  if t_quantity<=0 THEN
 11  RAISE_APPLICATION_ERROR(-20001,'the book is not available!');
 12  end if;
 13  end;
 14  /

Trigger created.

SQL>
SQL> --trying TRIGGER
SQL> update books set quantity=0 where book_id=100;

1 row updated.

SQL>  INSERT INTO Borrowing (borrow_id, member_id, book_id, borrowdate, duedate, returndate)
  2  VALUES (999, 101, 100, SYSDATE, SYSDATE+7, NULL);
 INSERT INTO Borrowing (borrow_id, member_id, book_id, borrowdate, duedate, returndate)
             *
ERROR at line 1:
ORA-20001: the book is not available!
ORA-06512: at "SYSTEM.UNAVAILABLEBORROW", line 8
ORA-04088: error during execution of trigger 'SYSTEM.UNAVAILABLEBORROW'
```

Queries for Book Borrowing Statistics and Author Search ;

```
SQL> --QUERIES--
SQL> select b.book_id,b.title,count(*) as timesborrowed
  2  from Borrowing b1
  3  join books b on b1.book_id=b.book_id
  4  GROUP by b.book_id,b.title
  5  ORDER by timesborrowed DESC;

   BOOK_ID TITLE                                                    TIMESBORROWED
---------- ---------------------------------------------------------------------
       100 The black cat                                                       1
       103 1984                                                                1
       101 think and grow Rich                                                 1
       102 the art of happiness                                                1

SQL> --before optimization
SQL> select * from books
  2  where author='George Orwell';

   BOOK_ID TITLE
---------- ---------------------------------------------------------------------
AUTHOR                                                        QUANTITY
------------------------------------------------------------ ----------
       103 1984
George Orwell                                                         2

SQL> --after optimization
SQL> select title
  2  from books
  3  where author='George Orwell';

TITLE
---------------------------------------------------------------------
1984
```

Before Optimization: SQL Query Explanation

```
SQL> --before optimization
SQL> explain plan for
  2  select m.member_id,m.f_name,m.L_name,b.duedate
  3  from Members m
  4  join Borrowing b on m.member_id=b.member_id
  5  where b.duedate<SYSDATE and b.returndate is null;

Explained.
```

Execution Plan for Index Creation :

```
SQL> select * from table(dbms_xplan.display);

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 4164117985

--------------------------------------------------------------------------------
------------

| Id  | Operation                    | Name         | Rows  | Bytes | Cost (%CPU)
| Time     |

--------------------------------------------------------------------------------
------------


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |              |     1 |    63 |     3   (0)
| 00:00:01 |

|   1 |  NESTED LOOPS                |              |       |       |
|          |

|   2 |   NESTED LOOPS               |              |     1 |    63 |     3   (0)
| 00:00:01 |

|*  3 |    TABLE ACCESS FULL         | BORROWING    |     1 |    31 |     2   (0)
| 00:00:01 |
```

Execution Plan Details :

```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------

|*  4 |    INDEX UNIQUE SCAN          | SYS_C009898 |      1 |        |      0   (0)
| 00:00:01 |

|   5 |   TABLE ACCESS BY INDEX ROWID| MEMBERS      |      1 |     32 |      1   (0)
| 00:00:01 |

--------------------------------------------------------------------------------
-------------


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   3 - filter("B"."RETURNDATE" IS NULL AND "B"."DUEDATE"<SYSDATE@!)
   4 - access("M"."MEMBER_ID"="B"."MEMBER_ID")

Note
-----
   - dynamic sampling used for this statement (level=2)

22 rows selected.
```

After Optimization: SQL Query Explanation :

```
SQL> --after optimization
SQL> explain plan for
  2  create index idx_borrowing_bookid on Borrowing(book_id);

Explained.

SQL> SELECT m.member_id, m.f_name, m.L_name
  2  FROM Members m
  3  WHERE EXISTS (
  4    select * FROM Borrowing b
  5     WHERE b.member_id = m.member_id
  6       AND b.duedate < SYSDATE
  7       AND b.returndate IS NULL
  8  );

 MEMBER_ID F_NAME     L_NAME
---------- ---------- --------------------
         3 Sara       Mohammed
```

Execution Plan for Nested Loops Join:

```
SQL> select * from table(dbms_xplan.display);

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 1378071944

--------------------------------------------------------------------------------
----------------

| Id  | Operation                | Name                      | Rows  | Bytes | Cost (%C
PU)| Time     |

--------------------------------------------------------------------------------
----------------

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
|   0 | CREATE INDEX STATEMENT |                           |    82 |  1066 |     3
(0)| 00:00:01 |

|   1 |  INDEX BUILD NON UNIQUE| IDX_BORROWING_BOOKID      |       |       |
   |          |

|   2 |   SORT CREATE INDEX    |                           |    82 |  1066 |
   |          |

|   3 |    TABLE ACCESS FULL   | BORROWING                 |    82 |  1066 |     2
(0)| 00:00:01 |
```

Estimated Index Size for SQL Execution Plan:

```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
---------------


Note
-----
   - estimated index size: 65536  bytes

14 rows selected.
```

Granting Privileges and Assigning Roles to User :

```
SQL> --grant privileges , assign role to user
SQL> GRANT ALL PRIVILEGES ON Books TO Admin;

Grant succeeded.

SQL> GRANT ALL PRIVILEGES ON Members TO Admin;

Grant succeeded.

SQL> GRANT ALL PRIVILEGES ON Borrowing TO Admin;

Grant succeeded.

SQL> GRANT ALL PRIVILEGES ON Users TO Admin;

Grant succeeded.

SQL> GRANT CREATE USER, ALTER USER, DROP USER, CREATE SESSION TO Admin;

Grant succeeded.

SQL> GRANT Admin TO wajd;

Grant succeeded.
```

Creating User and Role, Granting Privileges:

```
SQL> --MODERATOR--
SQL> --create USER , create Role
SQL>
SQL> CREATE USER Lina IDENTIFIED BY L02;

User created.

SQL> CREATE ROLE Moderator;

Role created.

SQL>
SQL> --grant privileges , assign role to user
SQL> GRANT SELECT,INSERT ,UPDATE ,DELETE ON Books TO Moderator;

Grant succeeded.

SQL> GRANT SELECT, UPDATE ON Members TO Moderator;

Grant succeeded.

SQL> GRANT SELECT ON Borrowing TO Moderator;

Grant succeeded.

SQL> GRANT Moderator TO Lina;

Grant succeeded.
```

Creating User and Role, Granting Privileges:

```
SQL> --MEMBER--
SQL> --create USER , create Role
SQL> CREATE USER Rawda IDENTIFIED BY Raw03;

User created.

SQL> CREATE ROLE Member;

Role created.

SQL>
SQL> --grant privileges , assign role to user
SQL> GRANT SELECT ON Books TO Member;

Grant succeeded.

SQL> GRANT SELECT, INSERT, UPDATE ON Borrowing TO Member;

Grant succeeded.

SQL> GRANT Member TO Rawda;

Grant succeeded.
```

Creating User and Role, Granting Privileges:

```
SQL> --GUEST--
SQL> --create USER , create Role
SQL> CREATE USER Reema IDENTIFIED BY R04;

User created.

SQL> CREATE ROLE Guest;

Role created.

SQL>
SQL> --grant privileges , assign role to user
SQL> GRANT SELECT ON Books TO Guest;

Grant succeeded.

SQL> GRANT SELECT ON Borrowing TO Guest;

Grant succeeded.

SQL> GRANT Guest TO Reema;

Grant succeeded.
```

Wajd user

User wajd deleting and Updating Records :

```
SQL> connect wajd
Enter password:
Connected.
SQL> delete from system.books where book_id = 107;

1 row deleted.

SQL> update system.members
  2  set email ='ahmed01@gmail.com'
  3  where member_id=01;

1 row updated.
```

Query to Retrieve Borrowing Records :

```
SQL> select borrow_id,member_id,book_id,borrowdate
  2  from system.borrowing;

 BORROW_ID  MEMBER_ID    BOOK_ID BORROWDAT
---------- ---------- ---------- ---------
      1134          1        100 01-APR-25
      1156          2        101 02-APR-25
```

Insert a New Book Record :

```
SQL> INSERT INTO system.Books VALUES (107, 'The Power of Now', 'Eckhart Tolle', 10);

1 row created.
```

Query to Retrieve Roles and names of users:

```
SQL> select username ,role
  2  from system.users;

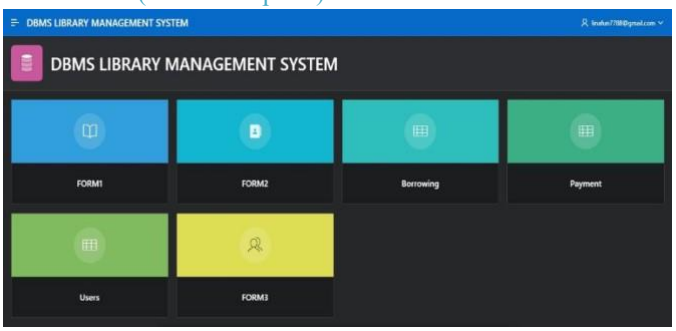
USERNAME
--------------------------------------------------
ROLE
--------------------------------------------------
Wajd
Admin

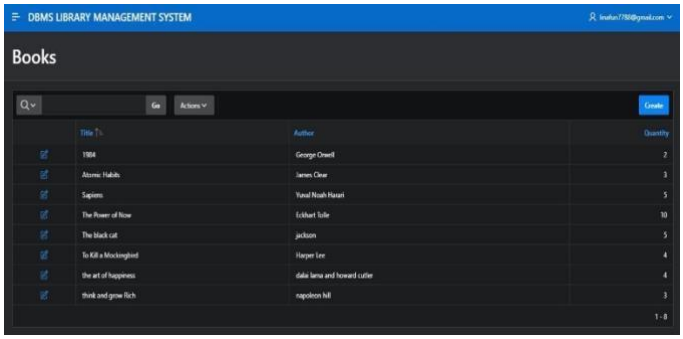
Lina
Moderator

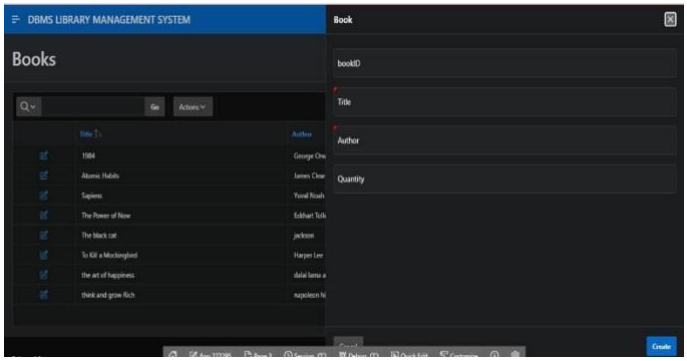
Rawda
Member

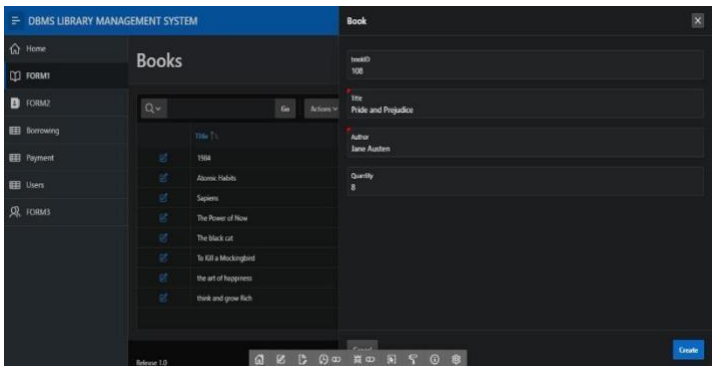

USERNAME
--------------------------------------------------
ROLE
--------------------------------------------------
Reema
Guest
```

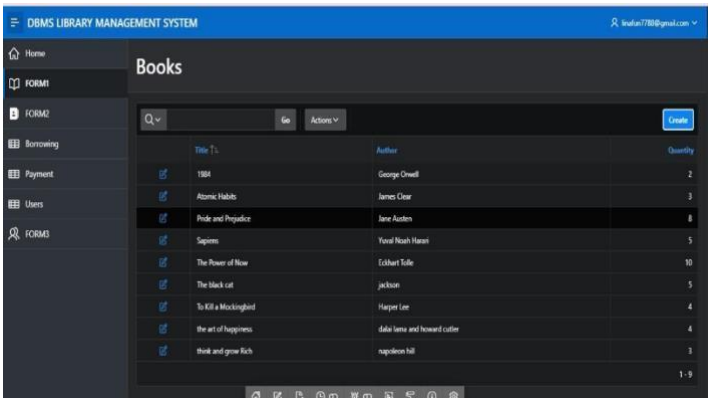
Interface (forms & reports) :

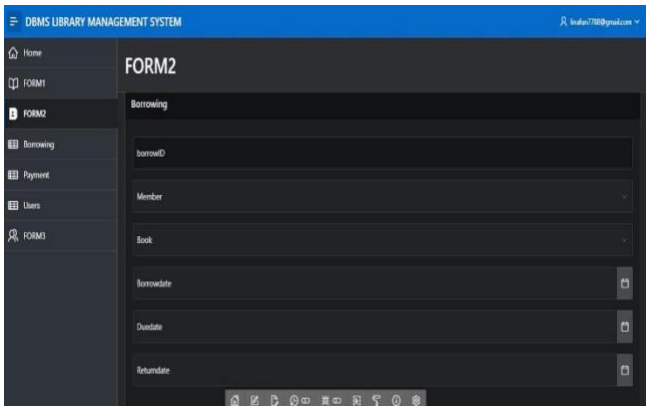


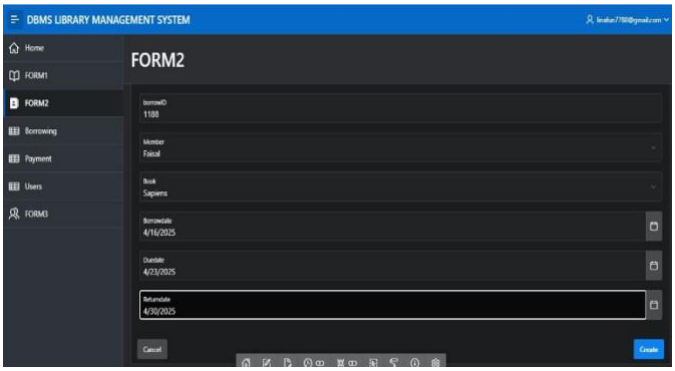


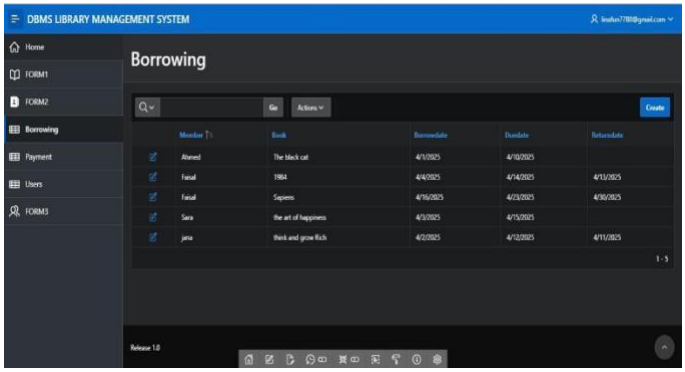


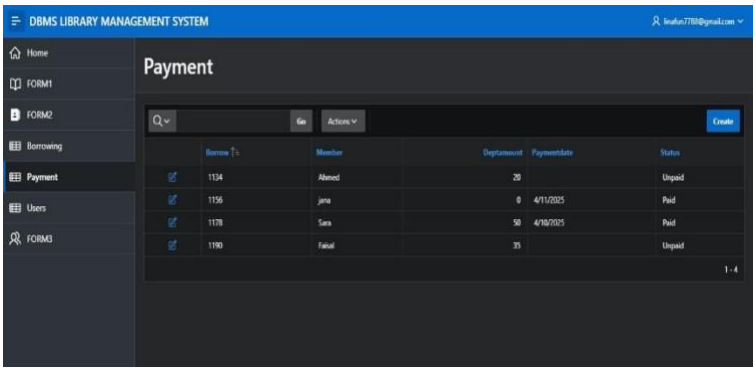


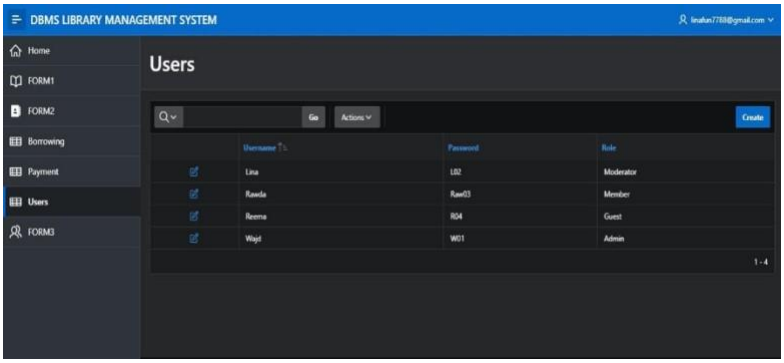


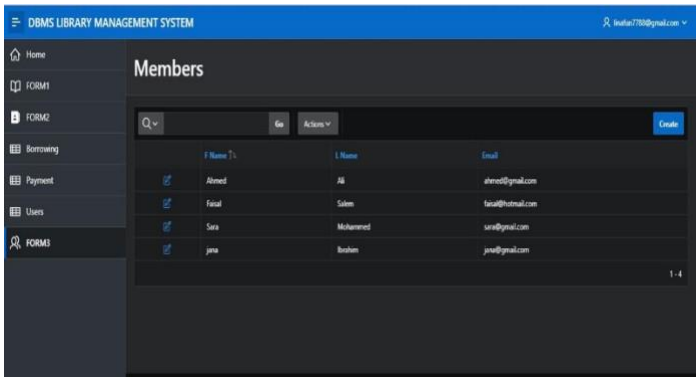


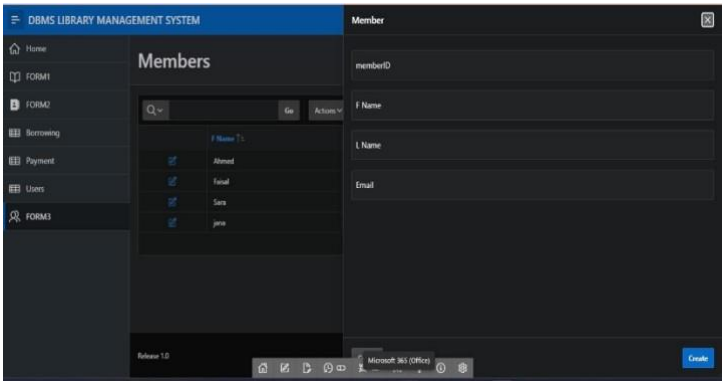


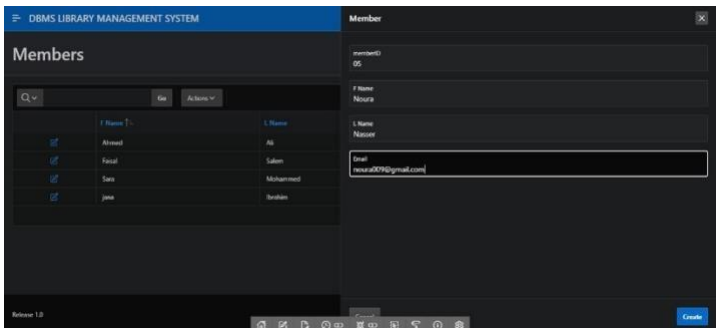


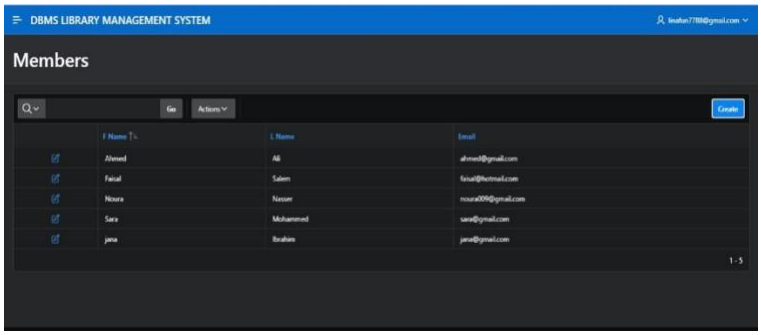












Interface Reference:

https://youtu.be/-nVrFyw1vZk?si=TQmCSK4B8u9TfNlB